

Building Omnichannel Retail Systems: Developer Insights from Modern OMS

Hanuman Reddy Gali
Andhra University
Northern VA CodeCamp Fall 2025

Presentation Agenda

1. The Omnichannel Challenge
2. Modern OMS Architecture
3. Intelligent Order Brokering
4. Fulfillment Operations
5. Integration & Data Sync
6. Performance & Scalability
7. Real-Time Operations & Configuration
8. Future Trends & Takeaways



The Omnichannel Challenge

Meeting Customer Expectations

Customers demand seamless shopping experiences across every touchpoint: from websites and mobile apps to marketplaces and physical stores.

They expect real-time inventory visibility and flexible fulfillment options, including BOPIS (Buy Online, Pick Up In Store) and ship-from-store.

Navigating Developer Realities

Developers wrestle with complex backend systems to unify inventory, orders, and critical customer data.

This often creates integration nightmares, especially when connecting legacy POS, diverse e-commerce platforms, and CRM systems.

OMS: From Backend Utility to Strategic Engine

1

Traditional OMS (Legacy Era)

Basic order processing and tracking. Limited integration. Manual workflows dominate.

2

Connected OMS (Digital Era)

Multi-channel support emerges. System integration begins. Inventory visibility improves.

3

Intelligent OMS (Modern Era)

AI-powered orchestration. Real-time decision making. Predictive fulfillment optimization.

4

Adaptive OMS (Future State)

Autonomous operations. Self-optimizing networks. Unified customer intelligence layer.

Core OMS Architecture Patterns: Building for the Future

API-First Design: Unlocking Agility

We build with **RESTful APIs** to provide flexible, efficient data retrieval for seamless integrations.

Our **microservices architecture** ensures independent scaling and resilience, adapting to your evolving business needs.

Event-Driven Workflows: Powering Real-time Responsiveness

We utilize **asynchronous messaging** to instantly propagate order state changes across your ecosystem.

Leveraging tools like **Kafka** we guarantee reliable, real-time event processing, keeping all systems in sync.

Ensuring Data Consistency: Integrity at Scale

We implement the **CQRS (**Command Query Responsibility Segregation) **pattern** to optimize performance and scalability by separating read and write operations.

Through **event sourcing**, we maintain a complete, immutable audit trail of every change, ensuring robust recovery and compliance.

Intelligent Order Brokering

Our intelligent system efficiently routes orders to the most optimal fulfillment locations. This is achieved by analyzing key factors through configurable business rules:

- **Customer Proximity:** Minimize delivery times and costs by prioritizing locations closest to the customer.
- **Real-time Inventory:** Confirm product availability instantly, preventing stock-outs and delays.
- **Cost & SLA Optimization:** Select fulfillment options that balance shipping expenses with critical service level agreements.
- **Operational Capacity:** Distribute orders intelligently across stores and warehouses to avoid bottlenecks and maximize efficiency.

These dynamic routing decisions are driven by a weighted algorithm. For instance, a basic routing engine might prioritize factors as follows:



Streamlining Modern Fulfillment Operations



Optimize Buy Online, Pick Up In Store (BOPIS)

Deliver a frictionless BOPIS experience, ensuring instant inventory synchronization, robust reservation systems, and automated customer notification workflows.



Expedite Curbside Pickup

Facilitate swift, contactless pickups with precise location-based triggers, seamless mobile check-in APIs, and rapid staff notification systems.



Enable Rapid Same-Day Delivery

Empower rapid same-day delivery through deep integration with leading logistics partners, intelligent route optimization, and comprehensive real-time tracking.



Unlock Endless Aisle Potential

Maximize sales opportunities with cross-channel inventory visibility, robust ship-from-store capabilities, and a truly unified customer shopping experience.



Integration Architecture

1

E-commerce Platforms

Seamlessly connect your online storefronts – like Shopify, Magento, or WooCommerce – through robust APIs to capture customer orders efficiently.

2

OMS Core

Drive efficient order processing with a powerful Order Management System that orchestrates workflows from capture to fulfillment.

3

Backend Systems

Integrate critical operational tools – including Point-of-Sale (POS), Enterprise Resource Planning (ERP), Warehouse Management (WMS), and Customer Relationship Management (CRM) – for unified data and streamlined operations.

Elevating Performance: Key Optimization Techniques

01

Optimize Your Database

Accelerate database performance by leveraging read replicas, intelligent connection pooling, and precise query optimization to manage high-volume transactions seamlessly.

02

Implement Smart Caching

Maximize application speed and responsiveness through strategic multi-layer caching. Integrate Redis for rapid data access, deploy CDNs for content delivery, and use intelligent invalidation to ensure data freshness.

03

Utilize Asynchronous Processing

Boost efficiency and user experience by offloading intensive tasks. Background job queues handle order fulfillment, real-time inventory updates, and customer notifications without slowing down your primary application.

04

Master Load Balancing

Ensure high availability and system stability with robust load balancing. Employ auto-scaling groups, proactive health checks, and circuit breaker patterns to distribute traffic and build resilience against failures.

Scaling for Success: Handling Peak Loads Effectively

Expand Horizontally for Growth

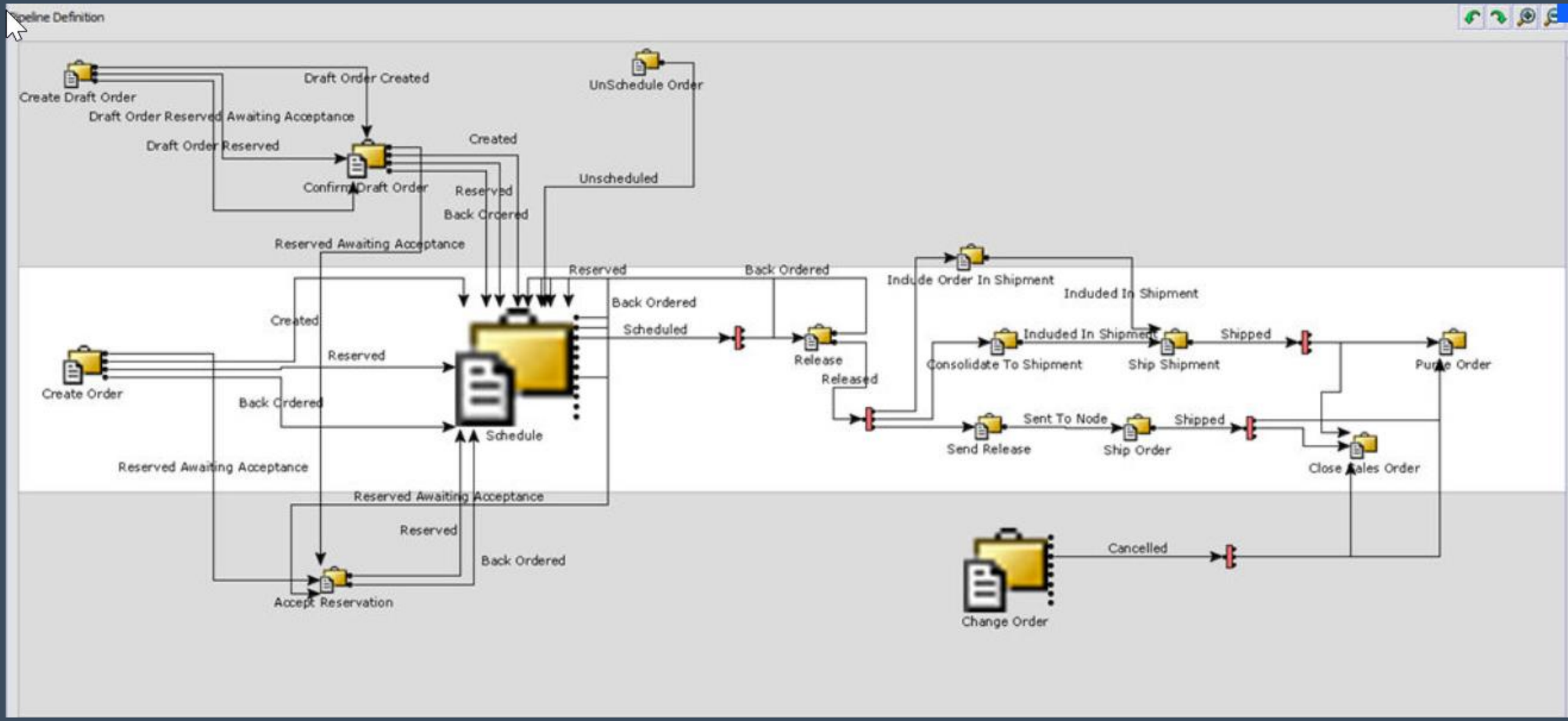
- **Deploy containerized microservices** using Kubernetes, allowing independent scaling of components like payment processing or user authentication.

- **Implement auto-scaling** that dynamically adjusts server capacity based on CPU usage or request queues, preventing slowdowns during traffic spikes.
- **Utilize database sharding** by distributing customer data across multiple database instances, significantly improving response times for large user bases.

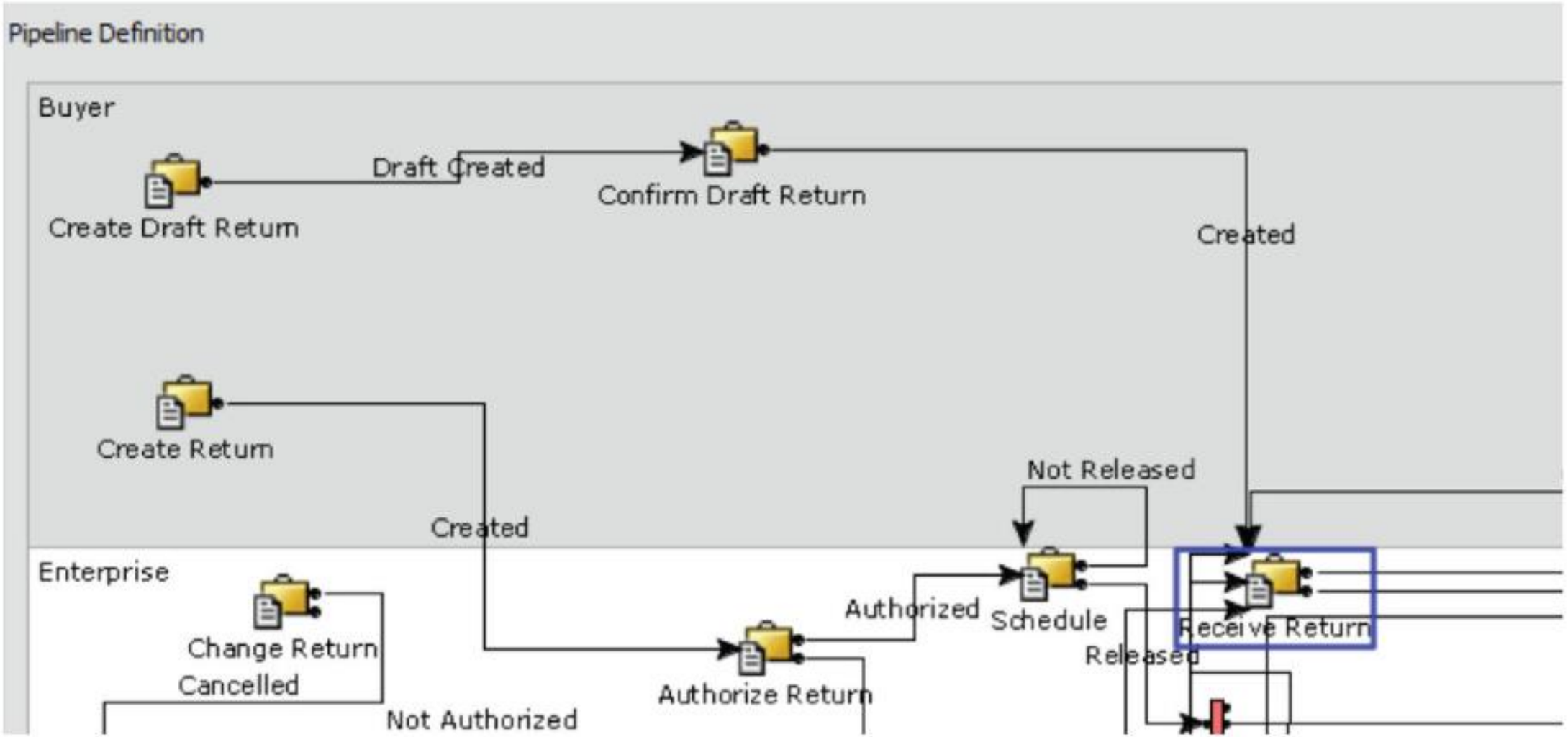
Optimize for Peak Performance

- **Introduce queue-based processing** for non-critical tasks like order fulfillment or notification emails, ensuring a smooth user experience even under heavy load.
- **Prioritize critical requests** for VIP customers or essential services, guaranteeing their access during high-traffic periods by implementing customer tiers (e.g., VIP, Gold, Platinum).
- **Design for graceful degradation** by temporarily disabling non-essential features during outages or extreme loads, maintaining core functionality.

Sales Order Process Flow



Return Order Process Flow



Create Order Sample

<Order EnterpriseCode="ACME" DocumentType="0001" EntryType="WEB" OrderNo="ORD12345">

<OrderLines>

<OrderLine ItemID="ITEM001" Quantity="1" UnitOfMeasure="EACH" DeliveryMethod="SHIP" FulfillmentType="SHIP_TO_HOME">
<<Item ItemID="ITEM001“ ItemShortDesc=“Test Desc”/>

<PersonInfoShip AddressLine1=“” City=“” State=“” ZipCode=“” Country=“”/>

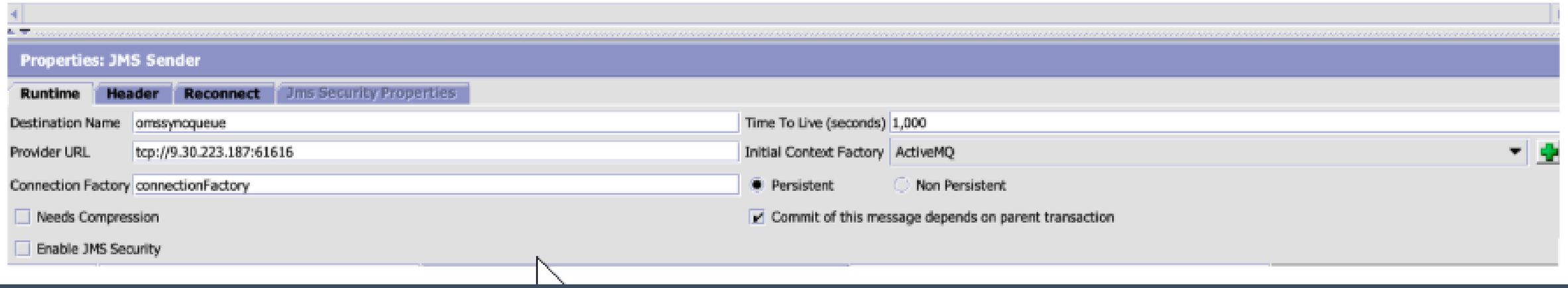
<PersonInfoMarkFor AddressLine1=“” City=“” State=“” ZipCode=“” Country=“”/>

<PaymentMethods>
<PaymentMethod CreditCardNo=“” CreditCardType=“” CreditCardExpDate=“” MaxChargeLimit=“”/>
</PaymentMethods>

</Charges>
</Taxes>
</Promotions>
</Notes>
<Instructions>
</OrderLineReservations>
</OrderLines>
<PersonInfoBillTo AddressLine1=“” City=“” State=“” ZipCode=“” Country=“”/>

Sync Service to drop message to MQ





ASync Service to read message from MQ



Properties: JMS Receiver

Runtime | Server | Reconnect | Exception | Exception References | Jms Security Properties

Sub Service Name	ActiveMQQueueReceiver	Destination Name	omsintqueue
Provider URL	tcp://9.30.223.187:61616	Initial Context Factory	
Connection Factory	connectionFactory	(•) Transactional	( ) Non Transactional
Initial Threads	1	Selector	
Service To Execute On EOF Message		Root Node Name Of EOF Message	
[ ] Enable JMS Security	[ ] Process Reply To Header	Default Reply To Destination	
[ ] Reply Correlation ID should be same as Request JMSMessage ID		Reply Time To Live (Seconds)	
[ ] Copy Header Properties To Reply Message			

Innovation Patterns & Future Trends

AI-Powered Predictions

Machine learning for demand forecasting and inventory optimization



Blockchain Integration

Supply chain transparency and authenticated product tracking

IoT Integration

Smart shelves, RFID tracking, and automated replenishment systems

Use Case: AI-Powered Order Risk Scoring in OMS

Integration Flow

1. **OMS receives order request** via `createOrder` API.
2. **Java service extracts order data** (e.g., customer ID, item, location).
3. **Java client sends data to AI model** (e.g., REST API or local ML model).
4. **AI returns risk score** (e.g., 0–1 scale).
5. **Java service decides**:
    1. If score < 0.3 → Proceed with order.
    2. If 0.3–0.7 → Flag for manual review.
    3. If > 0.7 → Hold or reject order.

```
    public static boolean isOrderSafe(String orderJson) throws Exception {
        URL url = new URL("https://ai-risk-api.example.com/evaluate");
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type", "application/json");
        conn.setDoOutput(true);

        try (OutputStream os = conn.getOutputStream()) {
            os.write(orderJson.getBytes());
        }

        InputStream responseStream = conn.getInputStream();
        Scanner scanner = new Scanner(responseStream).useDelimiter("\\A");
        String response = scanner.hasNext() ? scanner.next() : "";

        double riskScore = Double.parseDouble(response); // Assume response is a pla
        return riskScore < 0.3;
    }

    public static void main(String[] args) throws Exception {
        String sampleOrder = "{\"customerId\":\"C123\",\"itemId\":\"ITEM001\",\"zip\
        if (isOrderSafe(sampleOrder)) {
            System.out.println("Order is safe. Proceed.");
            // Call createOrder API here
        } else {
            System.out.println("Order flagged or held due to risk.");
        }
    }
}
```

Architectural Considerations for Enterprise Scale



Scalability Requirements

- Support for millions of daily order decisions
- Sub-second response times for inventory queries
- Elastic infrastructure for peak season demands
- Global deployment across geographic regions



Integration Complexity

- API-first architecture for system connectivity
- Event-driven messaging for real-time updates
- Master data management for inventory sync
- Legacy system accommodation strategies



Security and Compliance

- Customer data protection and privacy controls
- Payment card industry compliance standards
- Audit trails for fulfillment decisions
- Disaster recovery and business continuity

The Future of Algorithmically Powered Retail

As OMS platforms continue evolving, the convergence of artificial intelligence, IoT sensing, and autonomous systems will unlock unprecedented levels of operational efficiency and customer personalization.



Autonomous Fulfillment

Self-optimizing networks that adapt to changing conditions without human intervention



Predictive Positioning

AI models that pre-position inventory based on anticipated demand patterns



Hyper-Personalization

Fulfillment strategies tailored to individual customer preferences and behaviors

Key Takeaways for Developers

Start with API-first architecture

Design flexible, scalable interfaces that can adapt to changing business needs

Embrace event-driven patterns

Build resilient systems using asynchronous messaging and eventual consistency

Make configuration your competitive advantage

Externalize business logic to enable rapid feature deployment and A/B testing



Thank You!

Questions and Discussion

Hanuman Reddy Gali | Andhra University

Northern VA CodeCamp Fall 2025